vorrei un chiarimento riguardo all'uso di safe_load da parte del parser YAML python, si tratta di una feature non disponibile nella soluzione Perl ?

No: `safe_load` **non è una capacità assente in Perl.** È semplicemente il nome di una API specifica di PyYAML; l'equivalente concettuale in Perl è caricare YAML senza consentire la deserializzazione/blessing di oggetti, cosa che i modelli Perl moderni fanno già di default o permettono di controllare esplicitamente.[1] [2]

Cosa fa `safe_load` in Python

In PyYAML:

```
import yaml

with open("service.yaml", encoding="utf-8") as f:
    manifest = yaml.safe_load(f)
```

`safe_load()` usa `SafeLoader`, che accetta solo costrutti YAML standard — scalari, liste, mapping e tipi base — e **non** istanzia oggetti Python arbitrari tramite tag come `!!python/object` o `!!python/object/apply`.[3] [4]

Il rischio storico riguarda `yaml.load()` con loader unsafe:

```
# Da evitare per input non fidato
yaml.load(text)                       # deprecato/ambiguo nelle versioni recenti
yaml.load(text, Loader=yaml.Loader)   # unsafe
yaml.unsafe_load(text)                # unsafe
```

Questi possono deserializzare oggetti Python e, con tag appositi, eseguire codice. Per questo la raccomandazione è `yaml.safe_load()` oppure `yaml.load(text, Loader=yaml.SafeLoader)`.[4] [1]

Equivalente in Perl

In Perl il punto critico non è "eseguire codice Python", ma la possibilità che YAML crei oggetti Perl "blessed", cioè associati a una classe. Se un YAML non fidato può blessare un oggetto, può potenzialmente innescare comportamenti di classe, incluso `DESTROY`, durante la distruzione dell'oggetto.[2]

Modulo Perl	Comportamento sicuro
`YAML::XS`	Dalla versione 0.81, `$YAML::XS::LoadBlessed` è disabilitato per default: non carica oggetti blessed a meno che tu non lo abiliti esplicitamente. [2]
`YAML`	Dalla versione 1.30, `$YAML::LoadBlessed` è disabilitato per default. [2]
`YAML::Syck`	Dalla versione 1.32, `$YAML::Syck::LoadBlessed` è disabilitato per default. [2]
`YAML::Tiny`	Implementa un sottoinsieme di YAML orientato a configurazioni e strutture dati semplici; non è un parser YAML completo. [5] [6]
`YAML::PP`	Implementazione Perl pura con maggiore controllo su schemi, tag e sicurezza. [7]

Quindi, con una versione aggiornata di `YAML::XS`, questo è già l'equivalente prudente di `safe_load`:

```
use YAML::XS qw(LoadFile);

my $manifest = LoadFile('service.yaml');
```

Non vengono creati oggetti blessed a meno che tu non scriva esplicitamente:

```
local $YAML::XS::LoadBlessed = 1;
```

Il consiglio della documentazione/comunità Perl è di usare `local` in uno scope ristretto, così da non alterare il comportamento globale del processo. [2]

Differenza concettuale

La differenza è soprattutto **di API e modello di sicurezza**, non di capacità:

Aspetto	Python / PyYAML	Perl / YAML::XS
API sicura esplicita	`yaml.safe_load()`	`LoadFile()` con `LoadBlessed=0` di default
Meccanismo rischioso	Tag Python: `!!python/object`, `!!python/object/apply`	Blessing di oggetti Perl e tag/object serialization
Default storico	`yaml.load()` storicamente unsafe; PyYAML 5.1+ richiede loader esplicito e avvisa	Le librerie hanno reso `LoadBlessed=0` il default nelle versioni recenti
Controllo fine	Scelta del `Loader`: `SafeLoader`, `FullLoader`, `UnsafeLoader`	Variabile `LoadBlessed`; `YAML::PP` offre controlli più granulari
Raccomandazione pratica	`safe_load` per manifest e input esterni	`YAML::XS` aggiornato con `LoadBlessed` non abilitato; oppure `YAML::PP`

PyYAML distingue esplicitamente `SafeLoader`, `FullLoader` e `UnsafeLoader`; `SafeLoader` è quello raccomandato per input non fidato. In Perl, invece, la sicurezza è più spesso espressa come

“non abilitare il caricamento di oggetti blessed”, con default più sicuri nelle versioni recenti dei moduli principali. [1] [2]

Per `docker-dev`

Per i tuoi `services/*/service.yaml`, che sono file di configurazione locali ma potenzialmente provenienti da repository o template condivisi, userei:

Python

```
manifest = yaml.safe_load(stream)
```

Perl

```
use YAML::XS qw(LoadFile);

# Nessun local $YAML::XS::LoadBlessed = 1;
my $manifest = LoadFile($path);
```

E aggiungerei una validazione strutturale dopo il parsing in entrambi i casi: verificare che `manifest` sia una hashref/mapping e che contenga solo le chiavi attese. Il parser sicuro impedisce l’esecuzione o l’istanziazione pericolosa, ma non può sapere se `image`, `ports` o `healthcheck` hanno senso per il tuo dominio.

Quindi: `safe_load` **non è una feature mancante in Perl**; è il nome PyYAML per una politica di deserializzazione ristretta. In Perl ottieni lo stesso risultato con `YAML::XS` aggiornato e `LoadBlessed` disabilitato, oppure con `YAML::PP` se vuoi un controllo più esplicito e granulare. [7] [1] [2]

⁂

1. https://github.com/yaml/pyyaml/wiki/PyYAML-yaml.load(input)-Deprecation
2. https://blogs.perl.org/users/tinita/2020/01/making-yamlpm-yamlsyck-and-yamlxs-safer-by-default.html
3. https://www.sourcery.ai/vulnerabilities/python-pyyaml-unsafe-load-rce
4. https://codepathfinder.dev/registry/python/lang/PYTHON-LANG-SEC-041
5. https://metacpan.org/pod/YAML::Tiny
6. https://github.com/Perl-Toolchain-Gang/YAML-Tiny
7. https://github.com/ingydotnet/yaml-libyaml-pm/issues/45
8. https://backpan.perl.org/modules/by-authors/id/I/IN/INGY/Tiny-YAML-0.0.3.readme
9. https://blog.ankursundara.com/pyyaml-cve/
10. https://metacpan.org/dist/YAML-Tiny
11. https://pyyaml.org/wiki/PyYAMLDocumentation
12. https://nhimg.org/articles/pyyaml-api-safety-depends-on-version-and-loader-choice/
13. https://stackoverflow.com/questions/1773805/how-can-i-parse-a-yaml-file-in-python

14. https://www.chedong.com/phpMan.php/perldoc/YAML::XS/3pm

15. https://www.reddit.com/r/perl/comments/euvhxh/making_yamlpm_yamlsyck_and_yamlxs_safer_by_default/